

DESIGN AND SIMULATION OF A 32-BIT SINGLE-CYCLE MIPS CPU IN VHDL

ABSTRACT

This project focuses on the design and simulation of a 32-bit single-cycle MIPS-style CPU using VHDL. The system was built by creating the main processor components separately and then connecting them together into a complete datapath. The key modules include the ALU, register file, control unit, instruction memory, data memory, sign extension unit, and the CPU top level. Each component was tested using simulation testbenches and verified through waveform analysis in GTKWave. The project demonstrates how instructions are fetched, decoded, executed, and written back in a single-cycle processor architecture. The simulation results confirm that the system correctly performs arithmetic operations, memory access, register writes, control signal generation, and instruction fetching.

INTRODUCTION

The main goal of this project was to design a working 32-bit CPU in VHDL and verify it through simulation. The project is based on the idea of a simple MIPS-like single-cycle processor, where each instruction completes in one clock cycle. This kind of design is very useful because it clearly shows the relationship between instruction fetch, decoding, execution, memory access, and write-back.

*The project is relevant because it demonstrates core computer architecture concepts in a practical hardware description language environment. It also shows how a CPU can be divided into smaller modules, each with a specific task, and then integrated into a complete working system. The design process helped show how instructions such as add, sub, and, or, addi, lw, sw, beq, j, and jal can be supported using proper control signals and datapath connections.

The project also helped verify how a 32-bit datapath works in simulation, how the program counter behaves, and how instruction and data memories interact with the CPU.

ARCHITECTURE DESIGN

Overall Datapath

The CPU follows a single-cycle architecture. This means each instruction is fetched, decoded, executed, and completed in one clock cycle. The top-level CPU connects several modules together:

- Instruction Memory
- Control Unit
- Register File
- ALU Control
- ALU
- Data Memory
- Sign Extend unit

The instruction is fetched from instruction memory using the program counter. The opcode and instruction fields are then decoded by the control unit. The register file supplies the operand data. The ALU performs arithmetic or logical operations. If needed, the data memory reads or writes memory values. Finally, the result is written back to the register file.

Instruction Set

The CPU supports the following instructions used in the project:

- R-format instructions:
 - add
 - sub
 - and
 - or
 - slt
- I-format instructions:
 - addi
 - slti
 - lw
 - sw
 - beq

- J-format instructions:
 - j
 - jal

The control unit decodes the 6-bit MIPS opcode and generates the correct control signals for each instruction.

Components

1. Instruction Memory

The instruction memory stores the test instructions used for the simulation. It returns the instruction based on the program counter address. In the testbench, the instructions for Task 4 are shown at logical addresses 0, 2, 4, and 6.

2. Control Unit

The control unit interprets the opcode and generates all the control signals needed by the CPU. These signals include:

- reg_dst
- alu_src
- mem_to_reg
- mem_write
- mem_read
- branch
- jump
- reg_write
- sign_or_zero
- alu_op

3. Register File

The register file contains 8 registers in this project. It supports two simultaneous reads and one write. The testbench writes specific values into registers 1, 3, 5, and 7 and reads them back successfully.

4. ALU

The ALU performs arithmetic and logic operations:

- addition
- subtraction
- AND
- OR
- set on less than

5. Data Memory

The data memory stores 32-bit values and supports both read and write operations. It was tested by writing and reading back values at memory addresses 2 and 4.

6. Sign Extend

The sign extension module converts 16-bit immediate values into 32-bit values so they can be used by the CPU datapath.

7. CPU Top Level

The CPU top level connects all modules and handles the flow of instructions through the processor.

IMPLEMENTATION DETAILS

VHDL Design Methodology

The design was completed in a modular way. Each CPU component was coded separately, tested independently, and then integrated into the CPU top level. This method made debugging easier because each block could be verified before connecting it to the next one.

The design flow was:

1. Create each module in VHDL
2. Write a testbench for each module
3. Simulate each module separately
4. Check waveforms in GTKWave
5. Connect all modules into the CPU
6. Run the CPU simulation and verify the complete flow

Module Descriptions

ALU

The ALU accepts two 32-bit inputs and an ALU control signal. It generates a 32-bit result and a zero flag. The zero flag is used for branch instructions such as beq.

Register File

The register file supports read and write access to 8 registers. It stores 32-bit values and is used as the main source of operands for the CPU.

Control Unit

The control unit uses the 6-bit opcode to decide which datapath operations should be active. For example:

- add uses register inputs and writes the ALU result
- lw enables memory read and writes memory data back
- sw enables memory write
- beq enables branching
- jal writes the return address into \$ra

Instruction Memory

The instruction memory provides the instruction words used in the simulation. For Task 4, the four instructions were stored and verified in the waveform.

Data Memory

The data memory is used for load and store instructions. It supports 32-bit reads and writes.

CPU Top Level

The top-level CPU connects every module and allows the complete processor to be simulated as a working system.

Simulation Process

The project was simulated using GHDL and viewed in GTKWave. Each testbench was compiled and run separately. The resulting VCD files were opened in GTKWave to inspect the signals.

The main simulation files were:

- alu.vcd
- data_memory.vcd
- register_file.vcd

- control_unit.vcd
- instr_memory.vcd
- cpu.vcd

RESULTS AND DISCUSSION

Register File Results

The register file testbench successfully wrote the following values into registers 1, 3, 5, and 7:

- 1934858
- 8558447
- 203848544
- 20670420

The waveform showed the values being stored and then read back correctly. This confirms that the register file behaves as expected.

Instruction Memory Results

The instruction memory testbench correctly returned the Task 4 instructions:

- 00650820
- 00A72822
- 00A11824
- 00E32825

These correspond to:

- add \$t0, \$t1, \$t2
- sub \$t2, \$t2, \$t3
- and \$t1, \$t2, \$t0
- or \$t2, \$t3, \$t1

The waveform also showed the logical instruction addresses used for the task.

ALU Results

The ALU testbench verified the following operations:

- 2500 + 25000
- 540250 - 37800
- 53957 AND 30000
- 746353 OR 846465
- 58847537 compared with 72464383

The ALU produced the correct results for all operations, including the set-less-than comparison.

Data Memory Results

The data memory testbench verified:

- writing 1024 to memory address 2 and reading it back
- writing 429496 to memory address 4 and reading it back

The waveform confirmed that the memory stores and returns the expected 32-bit values.

Control Unit Results

The control unit waveform displayed the correct control signals for:

- add
- slti
- j
- jal
- lw
- sw
- beq
- addi

This confirms that the opcode decoding is working correctly and that the control unit generates the right signals for each instruction.

CPU Results

The CPU waveform showed the processor fetching and executing the instruction sequence correctly. The PC advanced in the expected 32-bit word-aligned pattern, and the instruction output matched the Task 4 instruction sequence. The CPU simulation demonstrated that the datapath integration works correctly.

Performance Evaluation

The CPU works as a single-cycle design, so each instruction completes in one clock cycle. This makes the design simple and easy to understand, although it is not optimized for speed like a pipelined processor. The simulation results show that the system is functionally correct and that all major components are working together.

CHALLENGES AND SOLUTIONS

1. Addressing Confusion in Instruction Memory

One challenge was the difference between logical instruction numbering and real byte-addressed PC values. The assignment used 0, 2, 4, 6 for instruction placement, while the CPU naturally uses 32-bit byte addresses such as 0, 4, 8, 12. This was handled by explaining the logical instruction numbering in the report and verifying the actual CPU behavior separately.

2. Control Signal Debugging

Another challenge was making sure the control unit generated the correct signals for each opcode. This was solved by creating a control-unit testbench and checking the waveform in GTKWave.

3. Register and Memory Verification

A third challenge was proving that the register file and data memory stored and returned the correct values. This was solved by using specific test values and confirming them in the waveform.

4. CPU Integration

Connecting all the modules together into one CPU was also a challenge. This was handled by testing every module individually before integrating them into the final CPU design.

CONCLUSION AND FUTURE WORK

Conclusion

This project successfully designed and simulated a 32-bit single-cycle MIPS-style CPU in VHDL. All the main components were implemented and verified through simulation. The results showed that the CPU can fetch instructions, decode them, perform ALU operations, access memory, and write results back correctly. The individual testbenches and waveform screenshots provided proof that the system works as intended.

Future Work

Possible improvements to the project include:

- expanding the register file beyond 8 registers
- adding more MIPS instructions

- improving the instruction memory size
- converting the design into a pipelined CPU
- adding better error handling and reset behavior
- making the memory system more realistic
- adding a more complete assembly program for testing

REFERENCES

Britton, R. L. (2004). *MIPS assembly language programming*. Pearson Prentice Hall.

GTKWave. (2023). *GTKWave fully featured wave viewer user's guide (Version 3.3)*.
<http://gtkwave.sourceforge.net/gtkwave.pdf>

Ashenden, P. J. (2008). *The designer's guide to VHDL* (3rd ed.). Morgan Kaufmann.

MIPS Technologies, Inc. (2001). *MIPS32 architecture for programmers volume II: The MIPS32 instruction set*.

Patterson, D. A., & Hennessy, J. L. (2017). *Computer organization and design: The hardware/software interface (MIPS Edition)* (5th ed.). Morgan Kaufmann.